

PROGRESSION · TIMELINE · GRAPH

state-chart

Native state machine diagram — states as a numbered list, transitions as nested inline-code refs.

SUBMISSION LIFECYCLE

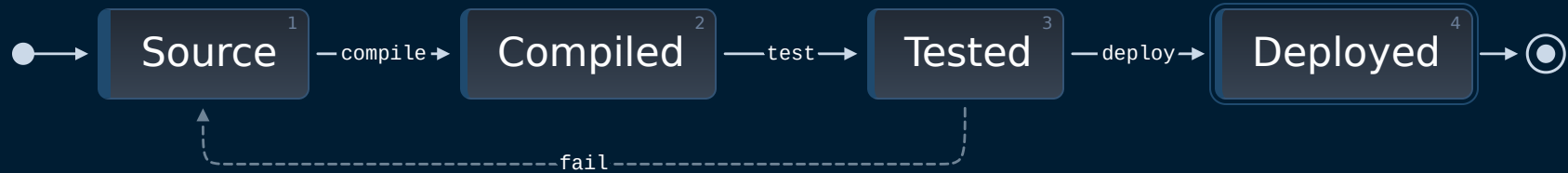
Document approval flow.

How a draft moves from author to publication.

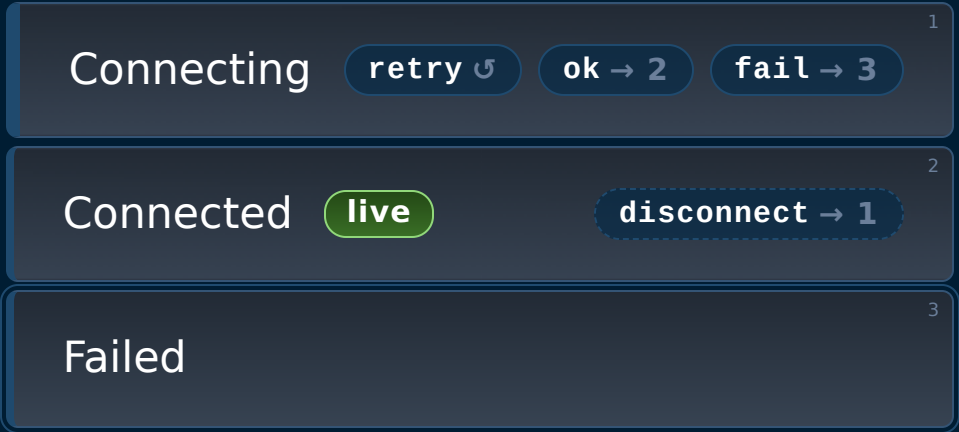


Rejected drafts return to the author; revisions stay in review.

Build pipeline.



Connection retry.

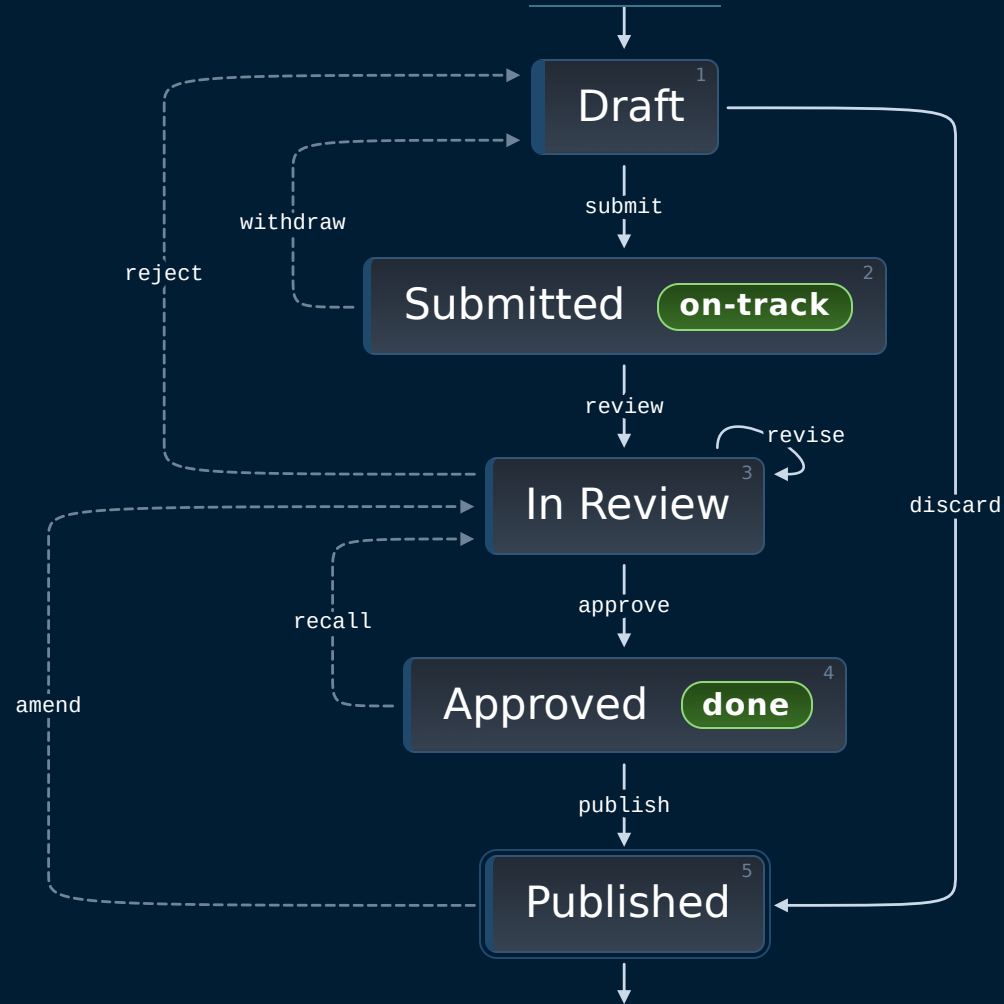


Document approval (curved).



SUBMISSION LIFECYCLE

Stress test — back-edges, skips, self-loop.



SUBMISSION LIFECYCLE

Document approval flow.

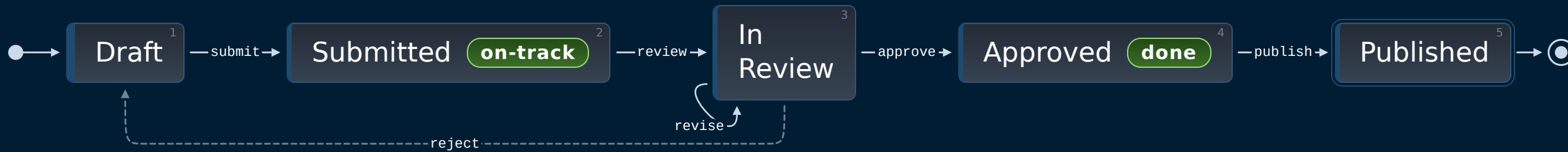
How a draft moves from author to publication.



Rejected drafts return to the author; revisions stay in review.

Document approval flow.

How a draft moves from author to publication.



Rejected drafts return to the author; revisions stay in review.

SUBMISSION LIFECYCLE

Document approval flow.

How a draft moves from author to publication.



Rejected drafts return to the author; revisions stay in review.

When NOT to reach for state-chart.

More than ~8 states. Vertical stacks of ten or more states stop reading as a machine and start reading as a list. If the system has many states, group them into phases and show one phase at a time, or step back to a higher-level abstraction. The chart's job is to make the topology obvious in one glance.

Hierarchical or parallel states. Composite states, orthogonal regions, history nodes — anything of states with one outgoing arrow per nested bullet. This layout doesn't — belong in a `diagram` component. Mermaid fence via

Continuous processes. If the diagram is really a workflow with stages that overlap or block (queue depth, throughput, capacity), a `gantt` or `kanban` chart reads better. State charts are for discrete, mutually-exclusive states the system flips between.

RELATED COMPONENTS

See also.

`diagram` — the machine has hierarchical states, parallel regions, or guards that need Mermaid's full state-diagram grammar

`journey` — the sequence is a user's path through tasks with mood / affect, not a system's discrete states

`timeline-list` — events are points in time rather than transitions between named states

`list-steps` — a linear procedure with no branching — state-chart is overkill if there are no choices to make

`roadmap` — parallel workstreams across phases, not a single machine's transitions